

Market BF

Marketplace locale et gestion commerciale
pour les commerces du Burkina Faso

JUSTIFICATION DES CHOIX TECHNOLOGIQUES

Node.js / Express • React + Vite • SQLite (Turso) • Docker / Render

Présenté par : NITIEMA Martial • [Nom du binôme]

Juillet 2026

1. Les critères qui ont guidé tous nos choix

Avant de choisir la moindre technologie, nous avons fixé quatre critères, directement liés au contexte du projet : une marketplace destinée aux commerçants et aux clients du Burkina Faso, développée en binôme, sans budget.

Critère	Ce que cela impose concrètement
Accessibilité maximale	L'application doit fonctionner sur n'importe quel téléphone ou ordinateur, sans installation, y compris avec une connexion modeste. Cela oriente vers le web plutôt qu'une application mobile native.
Coût d'infrastructure nul	Aucun service payant, aucune carte bancaire requise : hébergement, base de données, cartographie et stockage d'images doivent être gratuits.
Simplicité et productivité	Une équipe de deux personnes et un délai court : un seul langage sur toute la chaîne, des outils bien documentés, peu d'administration système.
Évolutivité	Les choix ne doivent pas bloquer l'avenir : pouvoir migrer vers PostgreSQL, brancher un agrégateur de paiement réel ou ajouter une application mobile sans tout refaire.

Chaque section qui suit applique cette grille : ce que fait la technologie dans Market BF, pourquoi elle a été retenue, et quelles alternatives ont été écartées.

2. Backend : Node.js + Express

Rôle dans le projet

Node.js exécute toute la logique serveur ; Express structure l'API REST en 9 modules de routes (authentification, boutiques, produits, stocks, commandes, avis, statistiques, administration, upload).

Pourquoi ce choix

- **Un seul langage partout** : JavaScript côté serveur comme côté client. En binôme, chacun peut intervenir sur tout le code sans changer de langage, de conventions ni d'outillage.
- **Léger et rapide à mettre en route** : un serveur Express tient en quelques lignes ; pas de structure imposée lourde, ce qui convient à un MVP.
- **Modèle non bloquant** : Node.js gère naturellement de nombreuses requêtes simultanées (consultations du catalogue, commandes) avec très peu de ressources — important sur un hébergement gratuit aux capacités limitées.
- **Écosystème npm** : tous les besoins du projet (JWT, bcrypt, validation, upload, e-mails) sont couverts par des bibliothèques éprouvées, installées en une commande.

Alternatives écartées

Django (Python) apportait un back-office intégré mais imposait un second langage et un cadre plus rigide pour une API simple. **PHP/Laravel** restait possible mais aurait également cassé l'unité JavaScript du projet. Express est le standard de facto pour une API REST Node.js, minimaliste et documenté partout.

3. Base de données : SQLite (better-sqlite3) puis Turso en production

Rôle dans le projet

La base contient les 9 tables du modèle (utilisateurs, boutiques, produits, stocks et mouvements, commandes et lignes, paiements, avis). En développement, c'est un simple fichier SQLite local ; en

production, la même base est hébergée chez Turso (libSQL, un SQLite accessible par le réseau).

Pourquoi ce choix

- **Zéro administration** : pas de serveur de base de données à installer, configurer, sauvegarder ; la base est créée et peuplée automatiquement au premier démarrage.
- **Transactions ACID** : SQLite garantit l'atomicité dont dépend la partie la plus sensible du projet — le décrétement de stock à la commande, protégé contre les commandes simultanées (pas de survente), et la restitution du stock en cas d'annulation.
- **Turso résout la persistance** : le plan gratuit de Render efface le disque à chaque redéploiement. En déplaçant la base chez Turso (gratuit, répliqué), comptes, produits et commandes survivent aux redémarrages — sans changer de moteur SQL.
- **Adapté à l'échelle réelle** : pour un MVP avec quelques dizaines de boutiques, SQLite est largement suffisant et plus rapide qu'un aller-retour vers un serveur SQL.

Alternatives écartées

PostgreSQL est plus puissant (concurrence en écriture élevée, types avancés) mais exige un serveur dédié — payant ou limité dans les offres gratuites — et une administration continue. Le SQL utilisé restant standard, une migration PostgreSQL est prévue si la charge l'exige. **MongoDB** a été écarté : nos données sont fortement relationnelles (une commande référence un client, une boutique, des produits, un paiement), ce qui plaide pour le relationnel et ses contraintes d'intégrité.

4. Frontend : React 18 + Vite, application web responsive

Rôle dans le projet

Le frontend est une single-page application de 16 pages réparties en trois espaces (client, commerçant, administrateur), construite avec React et servie en production sous forme de fichiers statiques.

Pourquoi le web plutôt qu'une application mobile

C'est le choix le plus structurant du projet. Une application web responsive est **immédiatement accessible** à tout possesseur de smartphone ou d'ordinateur au Burkina Faso : pas de téléchargement, pas de Play Store, pas d'espace de stockage requis, mises à jour instantanées pour tous. Un seul code à développer et maintenir à deux, au lieu d'un code mobile + un code web. L'API REST étant indépendante du client, une application mobile (PWA ou React Native) pourra s'y brancher plus tard sans rien réécrire côté serveur.

Pourquoi React et Vite

- **React 18** : découpage en composants réutilisables (cartes produit, notation par étoiles, barres de navigation), état géré proprement (panier, session), et le plus grand écosystème de bibliothèques du marché.
- **Vite** : serveur de développement quasi instantané et build de production optimisé (code minifié, découpé), sans configuration — un gain de productivité net face à des outillages plus anciens.
- **React Router** : routage côté client avec protection par rôle — un client ne peut pas atteindre les pages commerçant ou admin, en complément du contrôle serveur.

Alternatives écartées

Flutter produit de belles applications mobiles mais impose le langage Dart, un passage par les stores et un second code à maintenir. **Vue/Angular** sont des équivalents crédibles de React ; React a été préféré pour son écosystème (React-Leaflet, Recharts) et sa documentation.

5. Interface : Tailwind CSS, Leaflet, Recharts

Bibliothèque	Rôle	Pourquoi elle a été retenue
Tailwind CSS	Tout le style de l'application	Classes utilitaires directement dans les composants : design responsive (mobile d'abord) rapide à produire et cohérent, sans feuilles de style à synchroniser. Le build ne conserve que les classes utilisées (CSS final très léger).
Leaflet + OpenStreetMap (React-Leaflet)	Carte interactive des boutiques géolocalisées	Cartographie 100 % gratuite et open source, sans clé d'API ni carte bancaire — là où Google Maps exige un compte de facturation. Couverture correcte du Burkina Faso, bibliothèque légère, intégration React officielle.
Recharts	Graphiques des statistiques commerçant (CA, ventes, top produits)	Bibliothèque de graphiques pensée pour React (composants déclaratifs), gratuite, suffisante pour des courbes et histogrammes lisibles sans configuration complexe.
Axios	Appels HTTP vers l'API	Instance unique préconfigurée : URL de base et jeton JWT ajoutés automatiquement à chaque requête, gestion centralisée des erreurs (session expirée, etc.).
Lucide React / React Hot Toast	Icônes et notifications	Deux bibliothèques minimalistes qui améliorent l'expérience (retours visuels immédiats : produit ajouté, commande passée) sans alourdir l'application.

6. Authentification : JWT, bcrypt et Google Identity Services

Pourquoi JWT

L'API est **stateless** : à la connexion, le serveur délivre un jeton signé (HS256) contenant l'identité et le rôle de l'utilisateur ; chaque requête le présente dans l'en-tête Authorization. Aucune session à stocker côté serveur — ce qui simplifie le déploiement (le serveur peut redémarrer sans déconnecter personne) et permettra demain à une application mobile d'utiliser exactement la même API. Le middleware vérifie le jeton et le rôle (client / merchant / admin) avant chaque route protégée.

Pourquoi bcrypt

Les mots de passe ne sont jamais stockés en clair : bcrypt les hache avec un facteur de coût de 12, ce qui rend une attaque par force brute prohibitive même en cas de fuite de la base. C'est le standard recommandé, préféré à des hachages rapides (SHA-256 seul) précisément parce qu'il est lent.

Pourquoi Google Identity Services en complément

La connexion Google en un clic réduit la friction à l'inscription — beaucoup d'utilisateurs ont un compte Google sur leur téléphone Android, très répandu au Burkina Faso. Contrairement à une dépendance complète à Firebase Auth, notre authentification maison reste maîtrisée : Google n'est qu'un moyen d'identification supplémentaire, gratuit et optionnel (l'application fonctionne sans).

7. Services d'appui : Multer, Cloudinary, Nodemailer

Outil	Rôle	Pourquoi
Multer	Réception des images uploadées (produits, boutiques)	Middleware standard d'Express pour les fichiers ; il permet nos contrôles de sécurité : extension et type MIME vérifiés, 5 Mo maximum, nom de fichier aléatoire.
Cloudinary	Stockage des images en production	Le disque de Render étant effacé à chaque redéploiement, les photos doivent vivre ailleurs. Cloudinary offre un stockage gratuit, un CDN rapide et le redimensionnement à la volée. En local, simple dossier uploads/ sans compte requis.
Nodemailer + SMTP	Reçus de commande, confirmations de livraison, réinitialisation de mot de passe	Bibliothèque de référence pour l'e-mail en Node.js, compatible avec n'importe quel fournisseur SMTP (Gmail gratuit ici). Sans configuration SMTP, les e-mails sont simulés dans la console — pratique en développement.

8. Infrastructure : Docker, Nginx, Render, GitHub

Docker et Docker Compose

L'application est conteneurisée avec un build multi-étapes : une première étape construit le frontend avec Vite, la seconde produit une image Node unique qui sert à la fois l'API et les fichiers statiques. Résultat : **le même environnement partout** (machine de développement, Render, ou un futur VPS), plus de « ça marche chez moi », et un seul service à déployer — donc pas de configuration CORS entre un frontend et un backend séparés. Docker Compose permet aussi de lancer toute l'application en local en une commande.

Render + GitHub : le déploiement continu

Render construit et déploie automatiquement l'image Docker à chaque *git push* (blueprint render.yaml). Plan gratuit, HTTPS automatique, variables d'environnement pour les secrets (JWT, SMTP, OAuth) — jamais dans le code. Les limites du gratuit (mise en veille après 15 minutes, disque non persistant) sont

compensées par Turso et Cloudinary. Alternatives écartées : Heroku n'a plus d'offre gratuite ; un VPS (~5 €/mois) contredisait le critère de coût nul — le docker-compose du projet y fonctionnerait toutefois tel quel.

9. Vue d'ensemble

Besoin	Technologie retenue	Critère décisif
Serveur / API REST	Node.js + Express	Un seul langage, léger, écosystème npm
Base de données	SQLite → Turso (libSQL)	Zéro administration, ACID, gratuit et persistant
Interface utilisateur	React 18 + Vite	Accessible sans installation, composants, productivité
Style	Tailwind CSS	Responsive rapide et cohérent, CSS final minimal
Cartographie	Leaflet + OpenStreetMap	Gratuit, open source, sans clé d'API
Graphiques	Recharts	Composants React déclaratifs, gratuit
Authentification	JWT + bcrypt + Google	Stateless, sécurisé, connexion en un clic
Images	Multer + Cloudinary	Uploads contrôlés, stockage persistant gratuit
E-mails	Nodemailer + SMTP	Standard Node.js, fournisseur libre
Conteneurisation	Docker + Compose	Même environnement partout, un seul service
Hébergement	Render + GitHub	Déploiement continu gratuit, HTTPS automatique

10. Conclusion

Aucune de ces technologies n'a été choisie par habitude ou par effet de mode : chacune répond à au moins un des quatre critères de départ — accessibilité, coût nul, productivité en binôme, évolutivité. L'ensemble forme une chaîne cohérente : un seul langage (JavaScript) du navigateur à la base de données, un seul conteneur à déployer, et un coût d'infrastructure de 0 F CFA pour une application réellement en production.

Ces choix ne ferment aucune porte : le SQL standard autorise une migration PostgreSQL, l'API stateless accueillera une application mobile, et la brique de paiement simulée est prête à recevoir un agrégateur réel (Orange Money, Moov Money, Wave). La stack retenue est donc à la fois la plus adaptée au MVP d'aujourd'hui et un socle crédible pour la suite.